WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

Scenario:

The e-commerce system stores and manages information about different products and categories. Each product belongs to a specific category, and details such as product name, price, and available stock quantity are maintained.

The system also stores details about each category, including its name and description. This makes it easier to organize products based on their categories and manage product information efficiently.

PRODUCT AND CATEGORY TABLES

CATEGORY TABLE

```
Connected.
SQL> CREATE TABLE Category(Category_ID VARCHAR2(10),Category_Name VARCHAR2(25),Description VARCHAR2(60));

Table created.
```

PRODUCT TABLE

```
SQL> CREATE TABLE Product(Product_ID VARCHAR(10),Product_Name VARCHAR(20),Category_ID VARCHAR(10),Price NUMBER(9,2),Stock_Quantity Nu
mber);

Table created.
```

DEFINE RELATIONSHIPS AND CONSTRAINTS

RELATIONSHIP:

The Category and Product tables are connected through the Category_ID attribute.

One category can contain multiple products.

Each product belongs to one category.

Therefore, the relationship between Category and Product is One-to-Many (1:M).

Category_ID in the Product table acts as a Foreign Key and references Category_ID in the Category table.

CONSTRAINTS:

1. Primary Key:

Category_ID is the Primary Key in the Category table.

Product_ID is the Primary Key in the Product table.

2. Foreign Key:

Category_ID in the Product table is defined as a Foreign Key.

It references Category_ID in the Category table.

3. NOT NULL Constraint:

Category_Name in the Category table cannot be empty.

Product_Name in the Product table cannot be empty.

4. CHECK Constraint:

Price must be greater than 0.

Stock_Quantity must be greater than or equal to 0.

5. UNIQUE Constraint:

Category_Name must be unique.

Two categories cannot have the same category name.

PRIMARY KEY:

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);

Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);

Table altered.
```

FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FR_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

ADDING CONSTRAINT - not null, check, unique:

```
SQL>  ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;

Table altered.

SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;

Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.

SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

INSTERING VALUES:

CATEGORY TABLE:

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: Cat0001
Enter value for category_name: Electronics
Enter value for description: Electronics gadgets and devices
old   1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new   1: INSERT INTO Category VALUES('Cat0001','Electronics','Electronics gadgets and devices')

1 row created.
```

```
SQL> /
Enter value for category_id: Cat0002
Enter value for category_name: Clothing
Enter value for description: Men,Women and Kids Apparels
old   1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Descrip
tion')
new   1: INSERT INTO Category VALUES('Cat0002','Clothing','Men,Women and Kids
Apparels')

1 row created.
```

AFTER INSERTION CATEGORY TABLE:

```
SQL> SELECT * FROM Category;

CATEGORY_I
----------
CATEGORY_NAME
--------------------------------------------------
DESCRIPTION
--------------------------------------------------
Cat0001
Electronics
Electronics gadgets and devices

Cat0002
Clothing
Men,Women and Kids Apparels

CATEGORY_I
----------
CATEGORY_NAME
--------------------------------------------------
DESCRIPTION
--------------------------------------------------
```

PRODUCT TABLE:

```
SQL> INSERT INTO Product VALUES('&ProductID','&Product_Name','&CategoryID',&Price,&Stock_Quantity);
Enter value for productid: p1003
Enter value for product_name: Cycle
Enter value for categoryid: Cat0002
Enter value for price: 10000
Enter value for stock_quantity: 20
old   1: INSERT INTO Product VALUES('&ProductID','&Product_Name','&CategoryID',&Price,&Stock_Quantity)
new   1: INSERT INTO Product VALUES('p1003','Cycle','Cat0002',10000,20)

1 row created.
```

```
SQL> INSERT INTO Product VALUES('&ProductID','&Product_Name','&CategoryID',&Price,&Stock_Quantity);
Enter value for productid: p1002
Enter value for product_name: Smart Phones
Enter value for categoryid: Cat0001
Enter value for price: 15000
Enter value for stock_quantity: 10
old   1: INSERT INTO Product VALUES('&ProductID','&Product_Name','&CategoryID',&Price,&Stock_Quantity)
new   1: INSERT INTO Product VALUES('p1002','Smart Phones','Cat0001',15000,10)

1 row created.
```

AFTER INSERTING PRODUCT TABLE:

```
SQL> SELECT * FROM Product;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
p1001
Laptops
Cat0001         77000              2

p1002
Smart Phones
Cat0001         15000             10

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------

p1003
Cycle
Cat0002         10000             20
```

INSERT VALUE:

```
SQL> INSERT INTO PRODUCT VALUES ('P121','cot','Cat0002',19000,7)
;

1 row created.
```

```
SQL> UPDATE PRODUCT SET PRICE = 20000 WHERE PRODUCT_ID = 'P121';


1 row updated.

SQL> SELECT * FROM PRODUCT WHERE PRODUCT_ID = 'P121';

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
---------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- ---------------
P121
cot
Cat0002         20000              7
```

UPDATED VALUES:

```
SQL> UPDATE PRODUCT SET STOCK_QUANTITY = 10 WHERE PRODUCT_ID = '
P121';

1 row updated.

SQL> SELECT * FROM PRODUCT WHERE PRODUCT_ID = 'P121';

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
---------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- ---------------
P121
cot
Cat0002         20000             10
```

DELETED VALUES:

```
SQL> DELETE FROM PRODUCT WHERE PRODUCT_ID = 'P121';

1 row deleted.
```

```
SQL> SELECT * FROM PRODUCT;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
----------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------
p1001
Laptops
Cat0001         77000              2

p1002
Smart Phones
Cat0001         15000             10

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
----------------
CATEGORY_I      PRICE STOCK_QUANTITY
---------- ---------- --------------


p1003
Cycle
Cat0002         10000             20
```

COUNTING NUMBER OF PRODUCTS IN EACH CATEGORY:

```
SQL> SELECT C.CATEGORY_NAME, COUNT(P.PRODUCT_ID) AS PRODUCT_COUN
T FROM PRODUCT P JOIN CATEGORY C ON P.CATEGORY_ID = C.CATEGORY_I
D GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME
--------------------------------------------------------------------------------
---------------
PRODUCT_COUNT
-------------
Clothing
            1

Electronics
            2
```

FINDING THE HIGHEST-PRICED PRODUCT IN EACH CATEGORY:

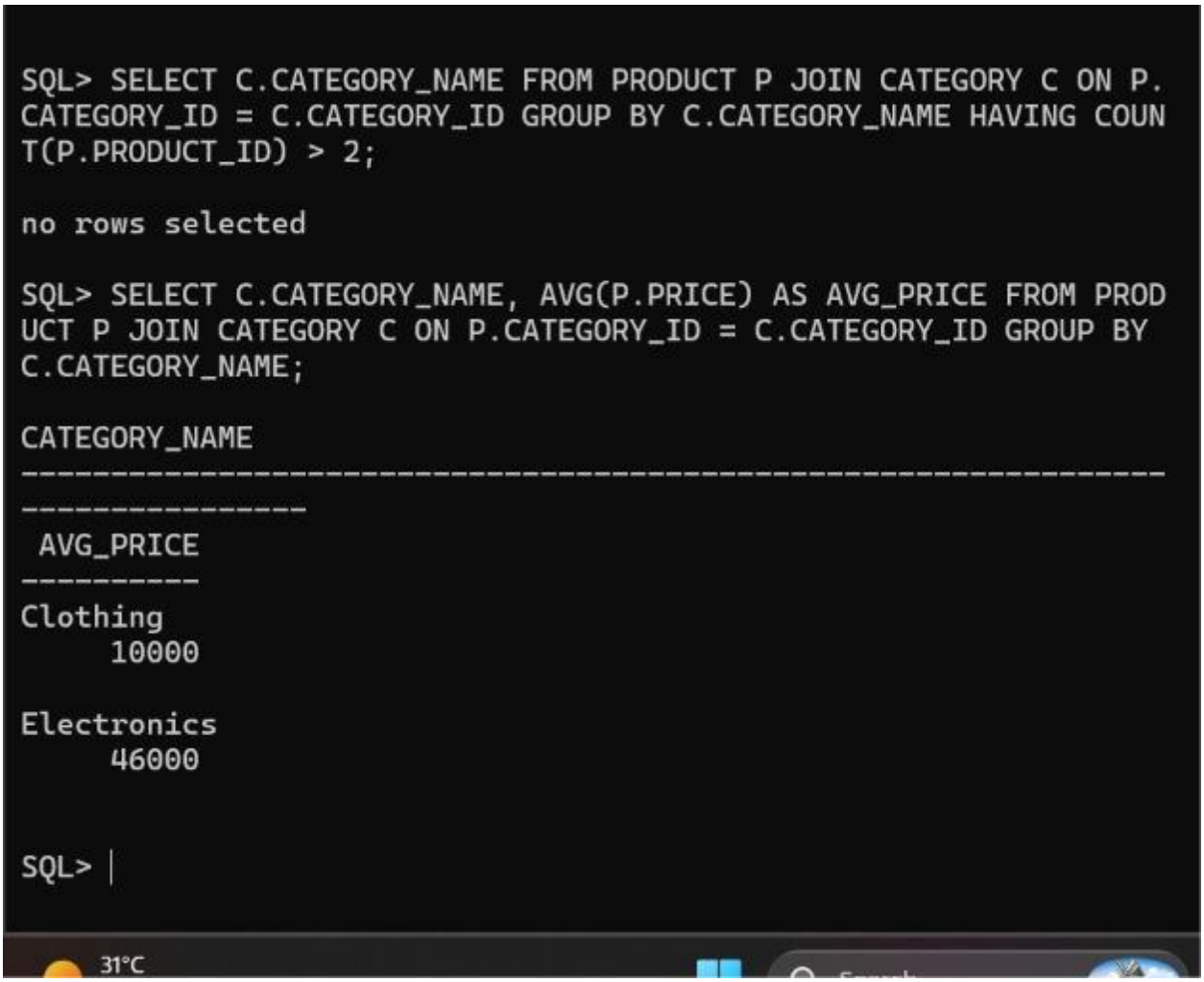
```
SQL> SELECT C.CATEGORY_NAME, P.PRODUCT_NAME, P.PRICE AS HIGHEST_
PRICE FROM PRODUCT P JOIN CATEGORY C ON P.CATEGORY_ID = C.CATEGO
RY_ID WHERE P.PRICE = (SELECT MAX(P2.PRICE) FROM PRODUCT P2 WHER
E P2.CATEGORY_ID = P.CATEGORY_ID);

CATEGORY_NAME
--------------------------------------------------------------------
---------------
PRODUCT_NAME
--------------------------------------------------------------------
---------------
HIGHEST_PRICE
-------------
Electronics
Laptops
        77000

Clothing
Cycle
        10000

CATEGORY_NAME
--------------------------------------------------------------------
---------------
PRODUCT_NAME
--------------------------------------------------------------------
---------------
HIGHEST_PRICE
```

FINDING THE AVERAGE PRODUCT PRICE CATEGORY-WISE:


SQL> SELECT C.CATEGORY_NAME FROM PRODUCT P JOIN CATEGORY C ON P.
CATEGORY_ID = C.CATEGORY_ID GROUP BY C.CATEGORY_NAME HAVING COUN
T(P.PRODUCT_ID) > 2;
no rows selected
SQL> SELECT C.CATEGORY_NAME, AVG(P.PRICE) AS AVG_PRICE FROM PROD
UCT P JOIN CATEGORY C ON P.CATEGORY_ID = C.CATEGORY_ID GROUP BY
C.CATEGORY_NAME;
CATEGORY_NAME
AVG_PRICE
Clothing
10000
Electronics
46000
SQL>
31°C